

Programmieren (T3INF1004)

DHBW Stuttgart

Christian Bader

christian.Bader@lehre.dhbw-stuttgart.de

Christian Alexander Holz

christian.holz@lehre.dhbw-stuttgart.de



Aufgabenstellung

Recherchieren Sie die folgenden Fragestellungen:

- Was ist CMake?
- Welches Problem löst CMake?
- Warum werden Build-Systeme benötigt?
- Welche Alternativen zu CMake gibt es?

Exkurs: CMake

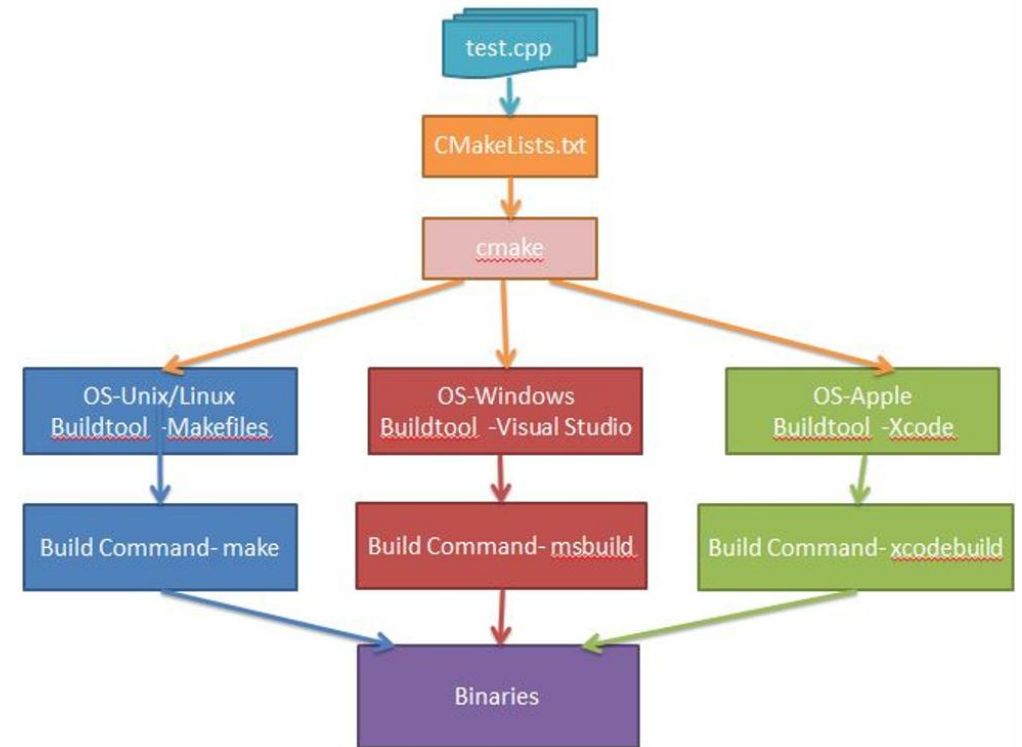
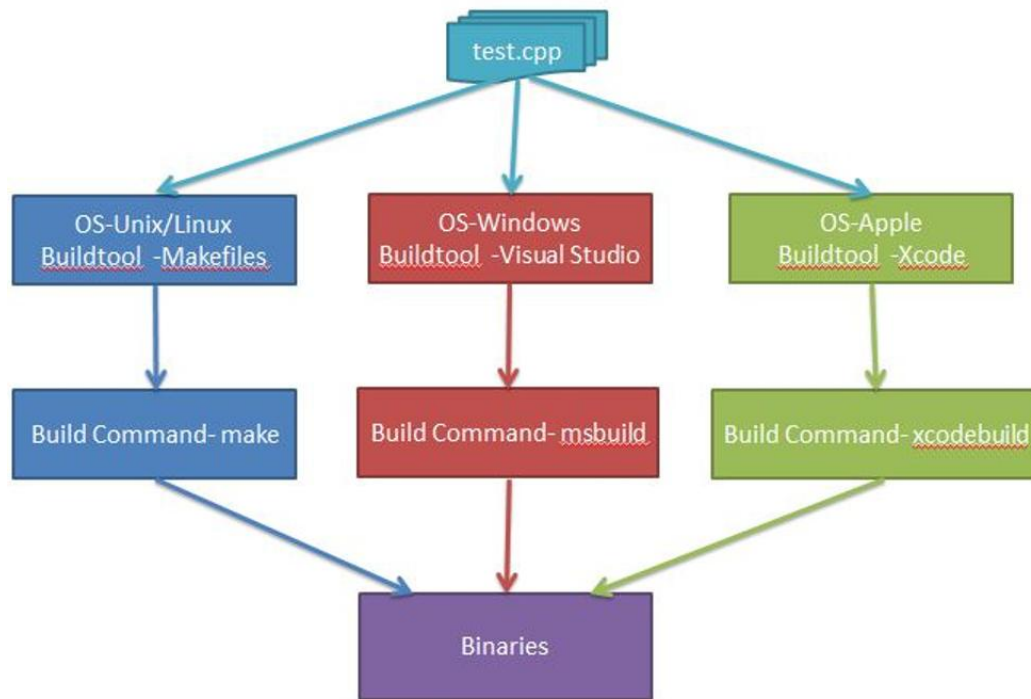
CMake - Überblick

- Verwaltung von Build-Prozessen in Softwareprojekten
- Generierung von Makefiles und Projektdateien
- Konfiguration des Projekts aus `CMakeLists.txt`
- Plattformübergreifendes Build-System
- Anwendung durch Konfigurations- und Buildphase
- Zum Bau von Libraries nützlich

→ Erstellung, Tests und Paketierung von Softwareprojekten



Cmake Prozess



CMakeLists.txt Beispiel

```
1  # Gibt die minimale CMake-Version an, die benötigt wird.
2  cmake_minimum_required(VERSION 3.10)
3
4  # Definiert das Projekt mit dem Namen "MyProject", der Version 1.0 und der verwendeten Sprache C++.
5  project(MyProject VERSION 1.0 LANGUAGES CXX)
6
7  # Setzt den C++ Standard auf C++17 und stellt sicher, dass dieser Standard zwingend erforderlich ist.
8  set(CMAKE_CXX_STANDARD 17)
9  set(CMAKE_CXX_STANDARD_REQUIRED True)
10
11 # Definiert das Verzeichnis, in dem OpenCV installiert ist.
12 set(OpenCV_DIR "C:/path/to/opencv")
13
14 # Findet die OpenCV-Pakete.
15 find_package(OpenCV REQUIRED)
16
17 # Definiert ein ausführbares Ziel namens "MyExecutable" und fügt die Quelldatei "main.cpp" hinzu.
18 add_executable(MyExecutable main.cpp)
19
20 # Definiert eine statische Bibliothek namens "MyLibrary" und fügt die Quelldatei "library.cpp" hinzu.
21 add_library(MyLibrary STATIC library.cpp)
22
23 # Gibt die Verzeichnisse an, die in der Bibliothek "MyLibrary" verwendet werden.
24 # Hier wird das Include-Verzeichnis relativ zum Quellverzeichnis des Projekts angegeben.
25 target_include_directories(MyLibrary PUBLIC ${CMAKE_SOURCE_DIR}/include)
26
27 # Verknüpft die Bibliothek "MyLibrary" mit dem ausführbaren Ziel "MyExecutable".
28 target_link_libraries(MyExecutable PRIVATE MyLibrary)
29
30 # Verknüpft die OpenCV-Bibliotheken mit dem ausführbaren Ziel "MyExecutable".
31 target_link_libraries(MyExecutable PRIVATE ${OpenCV_LIBS})
```

CMake - Kommandos

1. Build Ordner erstellen

```
mkdir build  
cd build
```

2. Phase 1: CMake Konfiguration (-G “MinGW Makefiles” nur für MinGW Compiler notwendig)

```
cmake -G "MinGW Makefiles" -DCMAKE_BUILD_TYPE=Debug ..
```

3. Phase 2: Build des Programms

```
cmake --build .
```

CMake - Weitere Features

- Setzen von Variablen mit `set(VARIABLE_NAME value)`
- Zugriff auf Variablen mit `${VARIABLE_NAME}`
- Control structures possible
 - `if/elseif/else/endif`
 - `foreach/endforeach`
 - `while / endwhile`
- Output Messages mithilfe
`message(<mode> „your message“)`
- Präprozessor Defines können gesetzt werden
`add_definitions(-DMY_DEFINE)`

```
set(LibSources src/car.cpp
  src/driver.cpp
  src/motorbike.cpp
  src/vehicle.cpp
)
add_library(vehicleLib STATIC ${LibSources})
target_include_directories(vehicleLib PUBLIC "${CMAKE_SOURCE_DIR}/inc/")
```

```
if(WIN32)
  message(STATUS "Compiling on Windows")
  add_definitions(-DOS_WINDOWS)
elseif(APPLE)
  message(STATUS "Compiling on macOS")
  add_definitions(-DOS_MAC)
endif()
```


Cmake - Einbinden von externen Libraries

1. Variante: mithilfe von `find_package()`
2. Variante: mithilfe von `FetchContent`

```
#Variante 1: --> Lokal & schnell
#set variable NLOHMAN_JSON_DIR to the directory
#set(NLOHMAN_JSON_DIR "${CMAKE_SOURCE_DIR}/3rd_Party/json/build/")
#message(STATUS ${NLOHMAN_JSON_DIR})
#help to find lib
#set(CMAKE_PREFIX_PATH ${NLOHMAN_JSON_DIR})
#Search for library
#find_package(nlohmann_json REQUIRED)

#Variante 2: --> einfach, kann im Build aber länger dauern
include(FetchContent)
FetchContent_Declare(json URL https://github.com/nlohmann/json/releases/download/v3.11.3/json.tar.xz)
FetchContent_MakeAvailable(json)
```